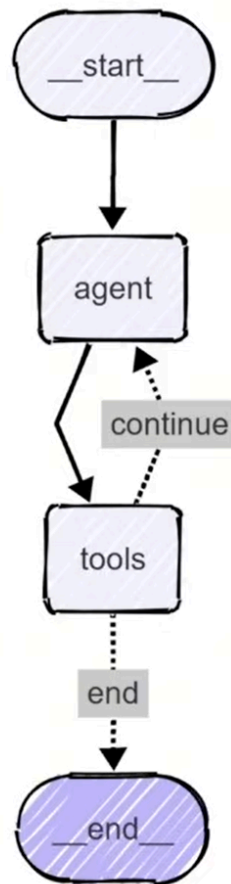


Human-AI Collaborative Drafting System

Task: Our company is not working efficiently! We spend way too much time drafting documents and this needs to be fixed!🕒 For the company, you need to create an AI Agentic System that can speed up drafting documents, emails, etc. The AI Agentic System should have Human-AI Collaboration meaning the Human should be able to provide continuous feedback and the AI Agent should stop when the Human is happy with the draft. The system should also be fast and be able to save the drafts.



Elements in the image

1. Oval labeled **__start__**

- This is the **entry point** of the workflow.
- The process always begins here.

2. Box labeled **agent**

- After the start, control goes to the **agent node**.
- This represents your function **our_agent**.
- The agent reads the current state, takes user input, and decides what to do.

3. Arrow from **agent** to **tools**

- Shows that after the agent responds, it may call a tool (like **update** or **save**).
- This path is taken when the AI decides it needs to execute an action.

4. Arrow back from **tools** to **agent**, labeled **continue**

- This shows a **loop**.

- If the conversation is not finished, after tool execution the process goes back to the agent to keep interacting.
- That means the user can give more instructions, and the AI can update the document again.

5. Arrow from **tools** to **end**, labeled **end**

- This path is taken when the user asks to **save the document**.
- The **should_continue** function checks the messages, and if it detects that the document was saved, it chooses this path.
- The workflow then finishes.

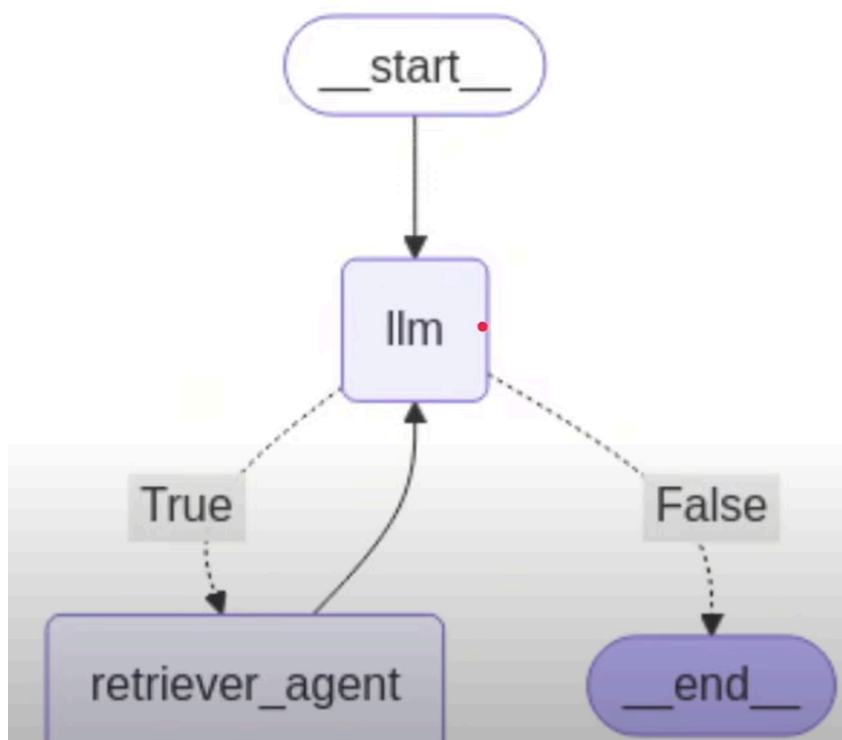
6. Oval labeled **__end__** (at the bottom, shaded purple)

- This is the **exit point** of the workflow.
 - Once here, the process is done and the Drafter stops running.
-

How the flow works

- Start → Agent asks the user what they want to do.
- Agent may call a tool like **update** or **save**.
- If **update** is called, after updating it loops back to Agent (continue working).
- If **save** is called, it exits and goes to End (process finishes).

RAG



Description of the Graph

1. Start Node (**__start__**)

- This is the **entry point** of the workflow.
- Think of it as the “power-on” button for the agent.
- The graph begins execution here.

2. LLM Node (**llm**)

- This node represents the **Language Model (LLM)** — the AI that talks to the user.
- The arrow from **__start__** points to **llm**, meaning the first action is to call the LLM.
- The LLM checks the user’s input and may decide whether it needs external information (a tool call).

3. Conditional Edges (True / False)

- After **llm** runs, there is a **decision point**:
 - **True**: LLM wants to use a tool (e.g., the retriever).
 - **False**: LLM doesn’t need a tool and can finish.
- These are represented by two dashed arrows labeled **True** and **False**.

4. Retriever Agent (**retriever_agent**)

- This node executes the **tool calls** made by the LLM.
- For example, it may search the PDF for relevant information.
- After executing the tool, it sends the results **back to the LLM**.
- Notice the arrow loops back from **retriever_agent** to **llm** — this forms a **feedback loop**.
- The LLM can now generate a more informed answer using the retrieved data.

5. End Node (**__end__**)

- If the LLM doesn't require any tool (False branch), the workflow goes directly to the **end**.
- This represents the **completion of the agent's task** — the final answer is ready.

How to Imagine It Without Seeing

- Think of it as a **conversation loop**:
 - Start → LLM receives your question.

- LLM decides: “Do I need to look something up?”
 - **Yes** → call retriever, get info, return to LLM.
 - **No** → go to end, answer is done.
- Loop repeats until the LLM has all the info it needs, then ends.
- It's like **asking a student a question**:
 - Student thinks, “Do I need to check my notes?”
 - If yes → goes to notes (retriever), comes back → answers.
 - If no → answers immediately.